Reviewer Pack — Minimal Rank of Coxeter Groups over Boolean Circuit Satisfia…

Entry 9b6dbfb340e4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 11:45:12 UTC

Minimal Rank of Coxeter Groups over Boolean Circuit Satisfiability

Entry ID: `9b6dbfb340e4` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-27 11:45:12 UTC

1. Conjecture

Field A (mathematical branch): Geometric Group Theory (Coxeter Groups) **Field B** (complexity object): Complexity Theory: Boolean Circuit Satisfiability

Statement:

> ['For every CNF formula F with n variables and m clauses, the minimal rank of the Coxeter group associated with the automorphism group of the vertex-labeled F is Θ(n log m).', 'Equivalently, for a given n and m, there exists an absolute constant c such that the number of generators of any Coxeter group representing the automorphism group of an F is at least cn log m.']

Rationale (proposer's reasoning):

> ['Coxeter groups are well-studied in geometric group theory and have connections to symmetries. The conjecture suggests that the complexity of Boolean circuit satisfiability might be related to the algebraic structure of these groups, potentially revealing new insights into NP-completeness.', 'If true, this could provide a novel way to analyze the complexity of circuits, possibly leading to new algorithms or theoretical results.']

Taxonomy category: `GEOMETRIC_GROUP_THEORY_TO_CIRCUIT_SATISFIABILITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `eabf7bfe53c69c54`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The minimal rank of the Coxeter group associated with the automorphism group of a vertex-labeled CNF formula is compared to Θ(n log m).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        variables = set(range(1, n + 1))
        clauses = []
        for _ in range(m):
            clause = random.sample(variables, random.randint(1, n))
            if random.choice([True, False]):
                clause = [-v for v in clause]
            clauses.append(clause)
        return clauses

    def compute_automorphism_group(cnf):
        # Simplified version of computing automorphism group
        # This is a placeholder and should be replaced with actual computation
        return set(range(1, len(cnf) + 1))

    def compute_coxeter_group_rank(group):
        # Placeholder for computing the rank of the Coxeter group
        # This is a simplified example and should be replaced with actual computation
        return len(group)

    n = random.randint(5, 40)
    m = random.randint(n, n * 10)  # Ensure at least as many clauses as variables
    cnf = generate_cnf(n, m)
    group = compute_automorphism_group(cnf)
    rank = compute_coxeter_group_rank(group)

    expected_min_rank = n * math.log(m)
```

```
    conjecture_holds = rank >= expected_min_rank

    return {
        "metric_name": "Coxeter Group Rank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"CNF: {cnf}, Expected Min Rank: {expected_m
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_d = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
        result = f"RESULT: SUPPORTED mean={mean_d} std=0 support_fraction={support_fraction}"
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = next((r["counterexample"] for r in results if not r["conjecture_holds"]),
        result = f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first

    print(result)
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bcfd918b.py", line 63, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bcfd918b.py", line 43, in run_trial
    cnf = generate_cnf(n, m)
          ^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bcfd918b.py", line 25, in generate_cnf
    clause = random.sample(variables, random.randint(1, n))
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   File "/usr/lib/python3.12/random.py", line 413, in sample
    raise TypeError("Population must be a sequence.  "
TypeError: Population must be a sequence.  For dicts or sets, use sorted(d).
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's support or falsification. | next: Investigate and fix the error in the test code to allow for proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11657
2	propose	ollama_remote	glm4:latest	0	0	10830
3	preregistration	ollama_remote	glm4:latest	0	0	5621
4	novelty	ollama_remote	glm4:latest	0	0	5064
5	novelty	ollama_remote	glm4:latest	0	0	5209
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11576
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7736
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9117
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7845
10	judge	ollama_remote	glm4:latest	0	0	12172

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 86827 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/9b6dbfb340e4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9b6dbfb340e4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/9b6dbfb340e4.tar.gz` (if generated)